PRASH-AI Study Material: Introduction to Transformers

1. Encoder-Decoder Architecture

An encoder-decoder model can take an input sequence of variable length and generate an output sequence of a different length. The encoder processes the input and creates representations. The decoder uses those representations to generate the output step by step.

2. The Context Bottleneck

In the original encoder-decoder approach, the input is compressed into a fixed-size context vector. For long inputs, this can become a bottleneck because too much information must fit into one representation.

3. Attention

Attention reduces the bottleneck by allowing the decoder to look at different encoder representations when generating each output token. It can focus more strongly on the parts of the input that are useful for the current step.

4. Self-Attention

Self-attention lets tokens in the same sequence consider one another. It helps the model understand relationships between words, even when they are far apart. It uses Query, Key, and Value representations to calculate how strongly tokens should influence one another.

5. Transformers

Transformers are neural-network architectures built around attention. They can process many tokens in parallel during training. Because attention does not automatically provide word order, Transformers use positional information.

6. Encoder vs Decoder

Encoder models are useful for understanding an input. Decoder models generate text token by token and commonly use causal or masked self-attention so they cannot look at future tokens. Encoder-decoder Transformers use both components.

7. Training Phase

During training, the model sees examples, makes predictions, calculates a loss, and updates its parameters using gradients and an optimizer. Language models commonly learn by predicting the next token from previous tokens.

8. Serving / Inference

After training, the model is used for inference. A user sends an input, the model processes it, and it generates a prediction or response. Serving makes the model available to an application or API.

Questions for PRASH-AI

1. Why is a fixed-size context vector a bottleneck?
2. How does attention reduce the bottleneck?
3. What is the difference between attention and self-attention?
4. What are Query, Key, and Value?
5. Why does a Transformer need positional information?
6. Why does a decoder use masked self-attention?
7. What is the difference between training and inference?